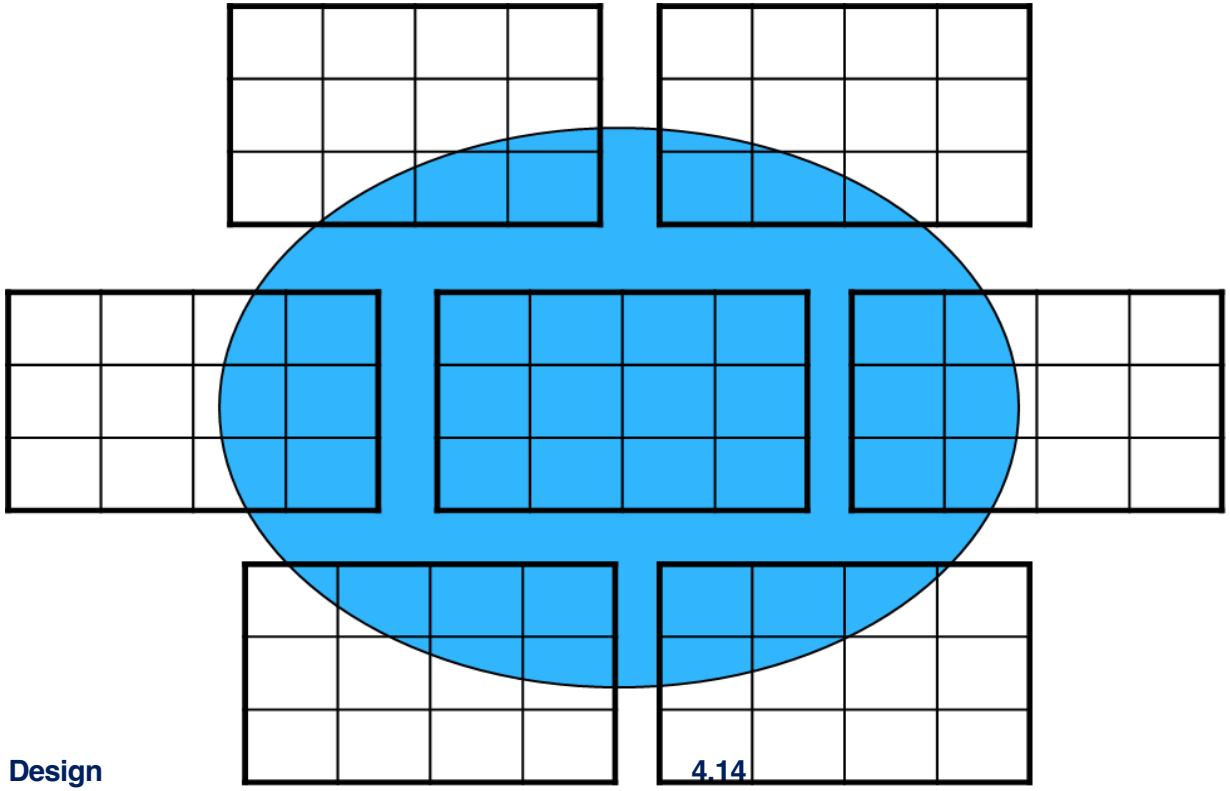
Chapter 4 : Intermediate SQL

4.2 Views

Database Principle and Design

4.2.1 View Definition

- A view is a virtual table defined by a **select** statement. This virtual table represents the data in the result of the **select** statement.
- A view can be used like a table in SQL statements
- The view does not itself contain a copy of the original data, but instead provides an alternative and simple way to access the original data
- The permanent table used to define the view is called base table



Create a View

- A view is defined using the **create view** statement which has the form

  create view *view_name* (A_1, ..., A_n) **as**
  select_statement

  - Attributes (A_1, ..., A_n) can be omitted and in that case the attributes in the select statement will be used.
  - Example: a view of instructors without their salary

  create view *faculty* **as**
  select *ID*, *name*, *dept_name* **from** *instructor*

- In SQL Server, a **create view** statement must be in a batch by its own

Internal Processing Logic

- Views in the database contain only definitions, not actual data.
- What does the DBMS do with the **create view** statement?
  - DBMS only stores the definition, it does not execute the **select** statement
  - Only when we retrieve data from the view, DBMS get the rows from the base tables according to the **select** statement
    - Example:

create view *faculty* **as**
select *ID*, *name*, *dept_name* **from** *instructor*

select * **from** *faculty* **where** *dept_name* = 'Physics'

- The above query is equivalent to the following query

select * **from** (**select** *ID*, *name*, *dept_name* **from** *instructor*) **as** *Faculty*

where *dept_name* = 'Physics'

Views Defined Using Other Views

- One view may be used to define another view

```
create view physics_fall_2017 as
   select section.course_id, sec_id, building, room_number
   from course, section
   where course.course_id = section.course_id
            and course.dept_name = 'Physics'
            and section.semester = 'Fall'
            and section.year = '2017';

create view physics_fall_2017_watson as
    select course_id, room_number
    from physics_fall_2017
    where building= 'Watson';
```

4.2.2 Benefits of Using Views in SQL Queries

- **Customize data**
  - Example:

    create view *physics_students* **as**

    select * **from** *student* **where** *dept_name* = 'Physics'
- **Focus on specific data**
  - Example:

    create view *dept_salary (dept_name, total_salary)* **as**
    select *dept_name*, **sum** (*salary*)
    from *instructor*
    group by *dept_name*;

Benefits of Using Views in SQL Queries (Cont.)

- **Combine partitioned data**
  - Example:

    create view *student_advisor (s_id, s_name, s_dept, i_id, i_name, i_dept)* **as**
    select *s_id, S.name, S.dept_name, i_id, I.name, I.dept_name*
    from *student* **as** *S*, *advisor* **as** *A*, *instructor* **as** *I*
    where *S.ID* **=** *A.s_id* **and** *I.ID* **=** *A.i_id*
- **Simplify data manipulation**
  - Example: find the advisor information of student "Zhang"

    select *i_id, i_name, i_dept*
    from *student_advisor* **where** *s_name* = 'Zhang'
  - Otherwise

    select *i_id, I.name, I.dept_name*
    from *student* **as** *S*, *advisor* **as** *A*, *instructor* **as** *I*
    where *S.ID* **=** *A.s_id* **and** *I.ID* **=** *A.i_id* **and** *S.name* = 'Zhang'
- **Enhance data security**
  - **View** provides a mechanism to hide certain data from the view of certain users (e.g, *physics_students, faculty*)

4.2.3 Materialized Views

- Certain database systems support **Materialized views** which contain actual data.
- **Materialized views** are updated whenever the underlying relations are updated.
- **Materialized views** are useful when the views are used frequently and the base tables are not changed frequently

4.2.4 Update of a View

- Although views are a useful tool for queries, they present serious problems if we express updates, insertions, or deletions with them.
- Example:
  - **create view** *instructor_info* **as**
    select *ID*, *name*, *building*
    from *instructor*, *department*
    where *instructor.dept_name= department.dept_name*;
  - **insert into** *instructor_info*
    values ('69987', 'White', 'Taylor');
  - This update cannot be translated uniquely.

Updatable Views in DBMS

- Different database systems specify different conditions under which they permit updates on views.
- In general, an SQL view is said to be updatable (i.e., inserts, updates, or deletes can be applied on the view) if the following conditions are all satisfied by the query defining the view:
  - The **from** clause has only one database relation.
  - The **select** clause contains only attribute names of the relation, and does not have any expressions, aggregates, or **distinct** specification.
  - Any attribute not listed in the **select** clause can be set to null
  - The query does not have a **group** by or **having** clause.
- Because of these conditions, modifications are generally not permitted on view in the application system.

4.3 Transactions

- A **transaction** is a collection of operations that performs a single logical function in a database application.
- Example: Raise all instructors' salaries by 10%

  update *instructor* **set** *salary = salary + salary * 10%*
- A transaction is a single unit of work, either all of its data modifications are performed, or none of them is performed.
- Each transaction must end with either a **commit** or **rollback** operation.
  - **Commit**: The updates performed by the transaction become permanent in the database.
  - **Rollback:** All the updates performed by the SQL statements in the transaction are undone.

Types of Transactions

- **Explicit transaction**
  - Used for multi-statement transactions
  - In SQL Server, explicitly start with the **begin transaction** statement and explicitly end with a **commit** or **rollback** statement
  - Example: transfer 100 dollars from Zhang's account to Wang's account

```
begin transaction
    update account set balance = balance -100
    where name = 'Zhang'
    update account set balance = balance +100
    where name = 'Wang'
commit
```

- **Implicit transaction (Auto-commit transaction)**
  - Each individual DML SQL statement, which is not in an explicit transaction, is a transaction on its own. The transaction begins implicitly when the SQL statement is executed, and end implicitly with either a **commit** or **rollback** operation.

4.4 Integrity Constraints

- **Integrity constraint** is a mechanism provided by DBMS to prevent data inconsistence caused by data manipulation.
- Integrity constraints are usually declared as part of the **create table** statement.
- In the **create table** statement, we can declare following constrains on a single table
  - **primary key**
  - **not null**
  - **unique**
  - **check** (P), where P is a predicate
  - **foreign key**

Primary Key Constraints

- Primary *keys can be* specified as part of the **create table** statement
- Examples:

```
create table department (
    dept_name varchar(20) primary key,
    building      varchar(15),
    budget        numeric(12,2)
);
```

```
create table classroom (
    building          varchar(15),
    room_number       varchar(7),
    capacity          numeric(4,0),
    primary key (building, room_number)
);
```

- The primary key attributes are required to be *non-null* and *unique*
- Although the primary key specification is optional, it is generally a good idea to specify a primary key for each relation.

4.4.2 Not Null Constraints

- **not null**
  - Declare *name* and *budget* to be **not null**

    name **varchar**(20) **not null**
    budget **numeric**(12,2) **not null**

4.4.3 Unique Constraints

- **unique** (A_1, A_2, …, A_m)
  - The unique specification states that the attributes A_1, A_2, …, A_m form a **candidate key**.
  - Candidate keys are permitted to be null (in contrast to primary keys).

4.4.4 Check Constraints (User defined Constraints)

- The **check** (P) clause specifies a predicate P that must be satisfied by every tuple in a relation.
- Example:

```
create table section (
    course_id varchar (8),
    sec_id varchar (8),
    semester varchar (6),
    year numeric (4,0) check (year > 1900),
    building varchar (15),
    room_number varchar (7),
    time slot id varchar (4),
    primary key (course_id, sec_id, semester, year),
    check (semester in ('Fall', 'Winter', 'Spring', 'Summer')))
```

4.4.5 Assigning Names to Constraints

- It is possible for us to assign a name to integrity constraints. Such names are useful if we want to drop or modify a constraint that was defined previously.
- To name a constraint, we precede the constraint with the keyword **constraint** and the name we wish to assign it. For example:

  constraint *minsalary* **check** (*salary* > 29000)

  - Later, if we want to modify this constraint, we can write:

    alter table *instructor* **drop constraint** *minsalary;*

    alter table *instructor* **add constraint** *minsalary* **check** (*salary* > 20000);

4.4.6 Referential Integrity Constraints

- Foreign keys can be specified as part of the **create table** statement
- Example:

```
create table teaches (
    ID                  varchar(5) references instructor,
    course_id           varchar(8),
    sec_id              varchar(8),
    semester            varchar(6),
    year                numeric(4,0),
    primary key (ID, course_id, sec_id, semester, year),
    foreign key (course_id, sec_id, semester, year) references section);
```

- By default, a foreign key references the primary-key attributes of the referenced table.
- SQL allows a list of attributes of the referenced relation to be specified explicitly.

  foreign key (*dept_name*) **references** *department* (*dept_name*)

  The specified list of attributes must, however, be declared as a candidate key of the referenced relation, using either a **primary key** constraint or a **unique** constraint.

Note: Foreign key *Constraint in SQL Server is different with one in relation model*

Cascading Actions in Referential Integrity

- A **delete** or **update** action on a referenced relation may violate a referential-integrity constraint, and such action is usually rejected.
- An alternative is to cascade **delete** or **update**:

```
create table instructor ( …
  dept_name varchar(20),
  foreign key (dept_name) references department
      on delete cascade
      on update cascade, …)
```

  - **on delete cascade:** delete tuples that reference the deleted tuples
  - **on update cascade:** update the attributes of foreign key in the referencing tuples to the new value

instructor relation

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

dept_name	*building*	*budget*
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

department relation

Cascading Actions in Referential Integrity (Cont.)

- Instead of cascade we can use :
  - **set null** (foreign key must be nullable)
  - **set default** (foreign key must have default value)

  Example:

  create table *instructor* (

  …

  dept_name **varchar**(20) **default** 'Music',

  foreign key (*dept_name*) **references** *department*

  on delete set null

  on update set default,

  …)

instructor relation

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

dept_name	*building*	*budget*
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

department relation

4.4.7 Integrity Constraint Violation During Transactions

- When a constraint is violated in a transaction, the normal procedure is to reject the SQL statement that caused the violation. There are two cases:
- If the SQL statement is a transaction on its own (i.e, implicit transaction), that transaction will be rolled back. For example:

insert into *instructor* **values** ('12121', 'Wu', 'Finance', '90000'),

('15151', 'Mozart', 'Music', '40000'), ('12121', 'Einstein', 'Physics', '95000'),

('32343', 'El Said', 'History', '60000');

- If the SQL statement is in an explicit transaction, only this statement is rejected. For example:

begin transaction

insert into *instructor* **values** ('12121', 'Wu', 'Finance', '90000');

insert into *instructor* **values** ('15151', 'Mozart', 'Music', '40000');

insert into *instructor* **values** ('12121', 'Einstein', 'Physics', '95000');

insert into *instructor* **values** ('32343', 'El Said', 'History', '60000');

commit

Catch Errors In A Transaction

- In SQL Server, a **try**…**catch** construct can be used to catch the runtime errors in a transaction.
- For example:

```
begin try
    begin transaction
        insert into instructor values ('12121', 'Wu', 'Finance', '90000');
        insert into instructor values ('15151', 'Mozart', 'Music', '40000');
        insert into instructor values ('12121', 'Einstein', 'Physics', '95000');
        insert into instructor values ('32343', 'El Said', 'History', '60000');
    commit
end try
begin catch
      print 'error catched:'
      print error_message()
      rollback
end catch
```

Avoid Integrity Constraint Violation During Transactions

- Consider *EMP* table :

```
create table EMP (
   EMPNO int,
   ENAME varchar(20) not null,
   JOB varchar(20),
   MGR int,
   HIREDATE date not null,
   SAL int,
   COMM int,
   DEPTNO int,
   check( SAL is null or SAL > 0),
   primary key (EMPNO),
   foreign key (DEPTNO) references DEPT  on delete set null,
   foreign key (MGR) references EMP)
```

- How to insert all employees without causing constraint violation
- Example:

```
insert into EMP (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,DEPTNO)
      values (7369,'SMITH','CLERK',7902,'1980-12-17',800,20)
```

Method 1

- Insert the employee's manager before inserting the employee

```
insert into EMP (EMPNO,ENAME,JOB,HIREDATE,SAL,DEPTNO)
        values (7839,'KING','PRESIDENT','1981-11-12',5000,10)
insert into EMP (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,DEPTNO)
        values (7655,'JONES','MANAGER',7839,'1981-4-2',2975,20)
insert into EMP (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,DEPTNO)
        values (7902,'FORD','ANALYST',7655,'1981-12-3',3000,20)
insert into EMP (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,DEPTNO)
        values (7369,'SMITH','CLERK',7902,'1980-12-17',800,20)
insert into EMP (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,DEPTNO)
        values (7698,'BLAKE','MANAGER',7839,'1991-5-1',2850,30)
insert into EMP (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,COMM,DEPTNO)
        values (7499,'ALLEN','SALESMAN',7698,'1981-2-20',1600,300,30)
insert into EMP (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,COMM,DEPTNO)
        values (7521,'WARD','SALESMAN',7698,'1981-2-22',1250,500,30)
insert into EMP (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,COMM,DEPTNO)
        values (7654,'MARTIN','SALESMAN',7698,'1981-9-28',1250,1400,30)
... ...
```

Method 2

- Set *MGR* to null initially, update after inserting all employees

insert into *EMP (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,DEPTNO*)
values (7369,'SMITH','CLERK',**NULL**,'1980-12-17',800,20)
insert into *EMP (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,DEPTNO*)
values (7902,'FORD','ANALYST',**NULL**,'1981-12-3',3000,20)
insert into *EMP (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,DEPTNO*)
values (7698,'BLAKE','MANAGER', **NULL**,'1991-5-1',2850,30)
insert into *EMP (EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,COMM,DEPTNO*)
values (7499,'ALLEN','SALESMAN', **NULL**,'1981-2-20',1600,300,30)
… …

update *EMP* **set** *MGR* = 7902 **where** *EMPNO* = 7369
update *EMP* **set** *MGR* = 7655 **where** *EMPNO* = 7902
update *EMP* **set** *MGR* = 7839 **where** *EMPNO* = 7698
update *EMP* **set** *MGR* = 7689 **where** *EMPNO* = 7499
… …

Method 3

- Insert all employees in one statement.

insert into *EMP (EMPNO,ENAME,JOB,HIREDATE,SAL,COMM,DEPTNO)*
values (7698,'BLAKE','MANAGER',7839,'1991-5-1',2850, **NULL**,30)
values (7499,'ALLEN','SALESMAN',7698,'1981-2-20',1600,300,30)
values (7521,'WARD','SALESMAN',7698,'1981-2-22',1250,500,30)
values (7654,'MARTIN','SALESMAN',7698,'1981-9-28',1250,1400,30)
values (7839,'KING','PRESIDENT','1981-11-12',5000,**NULL**,10),
values (7655,'JONES','MANAGER',7839,'1981-4-2',2975,**NULL**,20),
values (7902,'FORD','ANALYST',7655,'1981-12-3',3000,**NULL**,20),
values (7369,'SMITH','CLERK',7902,'1980-12-17',800, **NULL**,20)
… …

- The database system checks the constraints after all tuples are inserted.
- If a constraint violation occur, the insertion is rolled back, no tuples are inserted.

4.5 Built-in Data Types in SQL

Additional built-in data types supported by SQL

Database Principle and Design

4.5.1 Date and Time Types in SQL

- **date:** Dates, containing year, month and date
  - Example: **date** '2005-7-27'
- **time:** Time of day, in hours, minutes and seconds.
  - Example: **time** '09:00:30'
- **Datetime:** date plus time of day
  - Example: **datetime** '2005-7-27 09:00:30.75‘

Date and Time Functions

- SQL Server provides following date and time functions:
  - GETDATE(): returns the current system date and time
  - YEAR(date): returns an integer representing the year part of the specified date
  - MONTH(date): returns an integer representing the month part of the specified date
  - DAY(date): returns an integer representing the day part of the specified date
  - DATEDIFF(datepart, startdate, enddate): returns the number of 'datepart' units from startdate to enddate
    - **select** *ename*, **datediff**(**month**,*hiredate*,**getdate**()) **as** 'months' **from** *emp*

ename	months
SMITH	496
JONES	492
BLAKE	371
CLARK	490
SCOTT	421
KING	485
ADAMS	420
JAMES	484
FORD	484
MILLER	495

4.5.2 Type Conversion

- **Implicit type conversion:** When the types of two operands are different, SQL Server will try to convert the two operands to the same type. For examples:

ID char(5)

SELECT * **FROM** *instructor*
WHERE *id* > 9

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00

SELECT * **FROM** *instructor*
WHERE *id* > '9'

ID	name	dept_name	salary
98345	Kim	Elec. Eng.	80000.00

SELECT * **FROM** *course*
WHERE *course_id* > 9

Results Messages

```
Msg 245, Level 16, State 1, Line 2
在将 varchar 值 'BIO-101' 转换成数据类型 int 时失败。

Completion time: 2022-04-11T15:34:04.1233555+08:00
```

SELECT * **FROM** *course*
WHERE *course_id* > '9'

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3

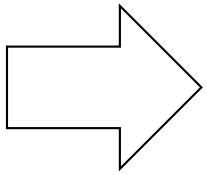
4.5.2 Type Conversion (Cont.)

- **Explicit type conversion: cast** (*e* **as** *t*)
  - Convert an expression *e* to the type *t*
  - *Example:* find instructors whose salaries start with "8"

select *name, salary* **from** *instructor*
where cast *(salary* **as** *varchar(10))* **like** *'8%';*

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

instructor relation



name	salary
Gold	87000.00
Singh	80000.00
Kim	80000.00

result

4.5.3 Default Values

- SQL allows a default value to be specified for an attribute in a relation. As a result, when a tuple is inserted into the relation, if no value is provided for the attribute, its value is set to the default value.
- Example

create table *student*
(ID **varchar** *(5),*
name **varchar** *(20)* **not null***,*
dept_name **varchar** *(20)* **references** *department,*
tot_cred **numeric** *(3,0)* **default** *0,*
primary key *(ID));*

```
insert into student(ID,name,dept_name)
values ('99999', 'Peltier', 'Physics')
```

4.5.4 Large-Object Types

- Photos, videos and audios are stored as a *large object*:
  - **blob**: binary large object -- object is a large collection of uninterpreted binary data (whose interpretation is left to an application outside of the database system)
  - **clob**: character large object -- object is a large collection of character data
- For example, we may declare attributes

  book_review clob(10KB)

  image blob(10MB)

  movie blob(2GB)
- When a query returns a large object, a pointer is returned rather than the large object itself. The application program can use this pointer to retrieve large objects.

Note: SQL server does not support BLOB and CLOB data types, but provides other complex data types (e.g, image, binary, text) for multimedia data

4.5.5 User-Defined Types

- **create type** statement in SQL Server creates user-defined type

  create type *Dollars* **from numeric (12,2) not null**

- Example

  create table *department* (
  dept_name **varchar** (20),
  building **varchar** (15),
  budget Dollars
  );

- **drop type** statement to drop types that have been created earlier.

4.5.6 Generating Unique Key Values

- Managing unique key values can sometimes be difficulty.
- Many DBMS offer automatic management of unique key-value generation. In SQL Server, **identity** can be used to define a attribute as a unique key whose value is generated by the system.
- For example, *ID* in *instructor* relation can defined as

  ID int **identity**

  When inserting a new instructor, we do not need specify a value for ID in **insert** statement, DBMS will generate this value automatically

  insert into *instructor* (*name*, *dept_name*, *salary*)

  values (‘Smith', 'Comp. Sci.', 100000);

4.5.7 Common errors in creating tables

char(4) is better

```
create table EMP (
    EMPNO varchar(4),
    ENAME varchar(20),
    JOB varchar(20),
    MGR varchar(4),
    HIREDATE varchar(20),
    SAL varchar(20),
    COMM varchar(20),
    DEPTNO varchar(2),
);
```

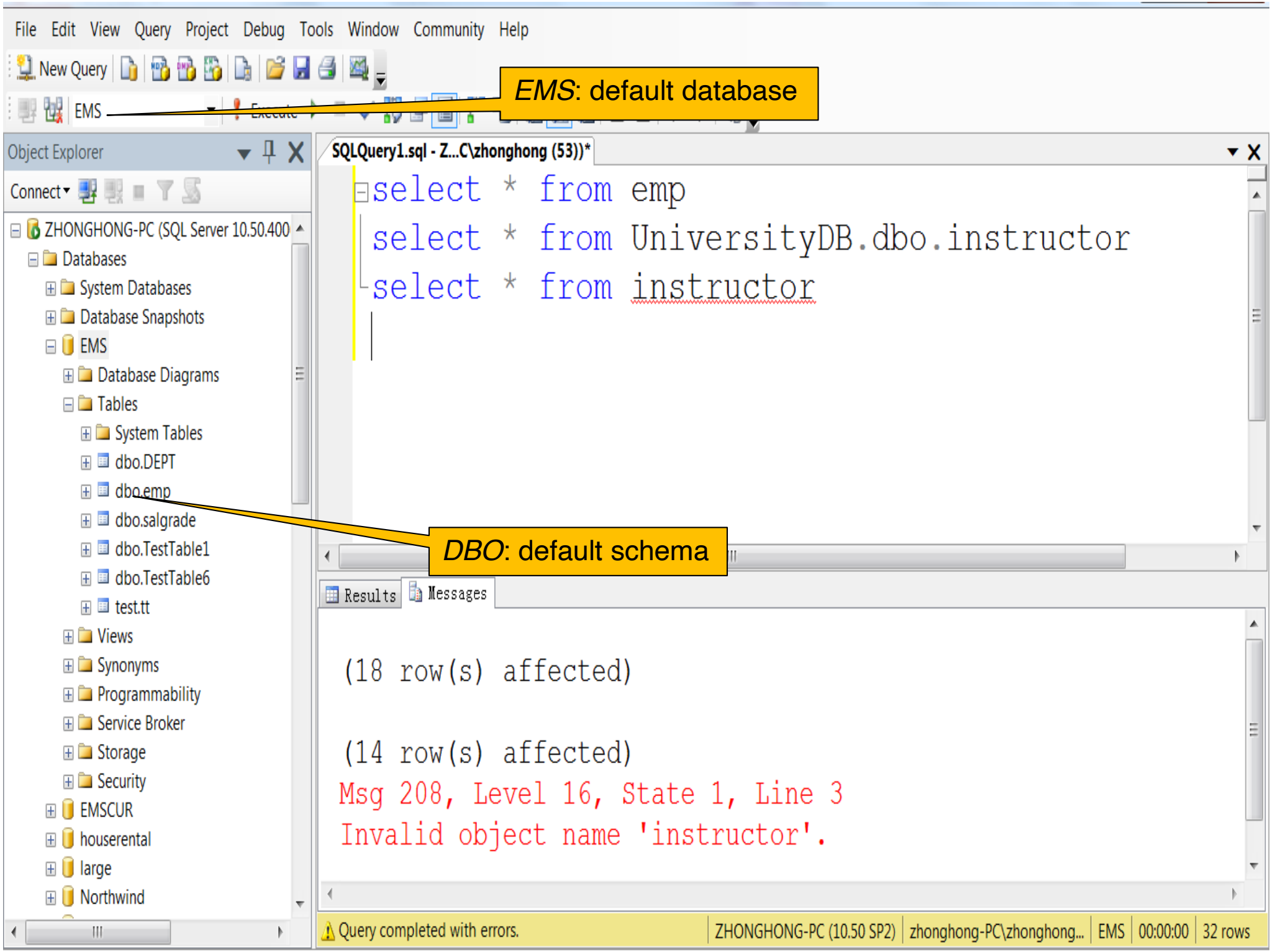
Inappropriate data types can lead to potential data errors, unnecessary type conversions, and calculation errors

```
create table EMP (
    SAL int,
    COMM int,
    DEPTNO int,
    check( SAL is null or SAL > 0),
    check( COMM is null or COMM > 0),
    primary key (EMPNO),
    foreign key (DEPTNO) references DEPT
```

Lack of constrains. Although constraints can slow down data modification, they can help us ensure data consistency and avoid program bugs and input errors

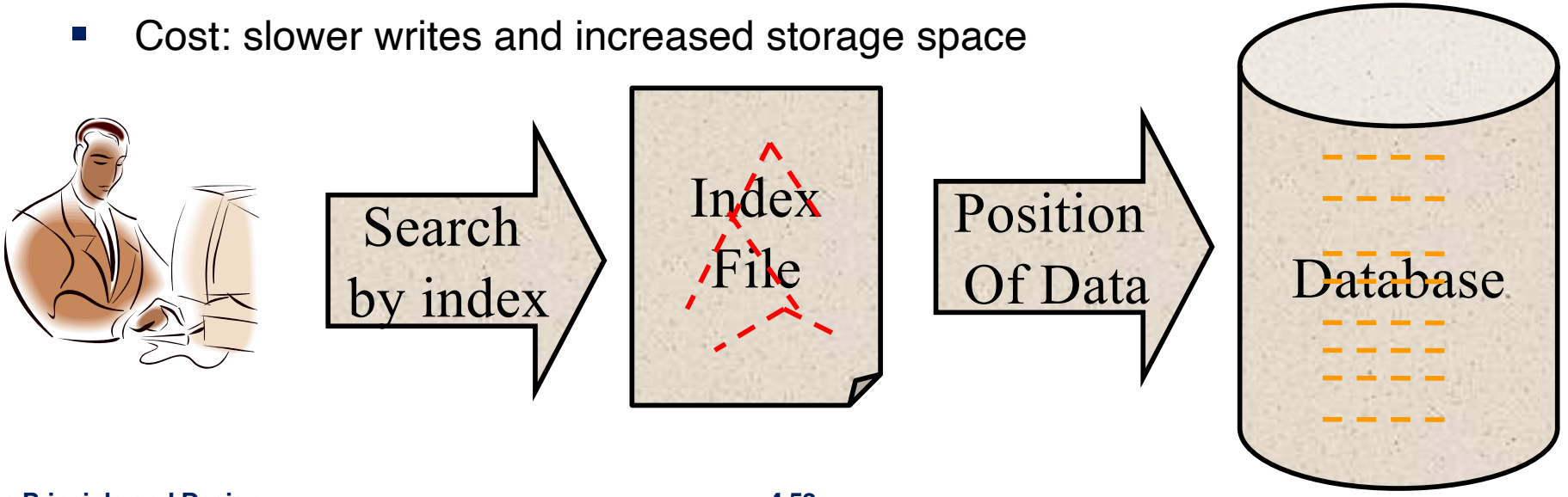
4.5.8 Schemas, Catalogs, and Environments

- Contemporary database systems provide a three-level hierarchy (namespace) for naming objects: **catalog-schema-object**
- **Catalog**: A database system may contain multiple catalogs. (SQL Server use the term *database* in place of the term *c*atalog.)
- Creation and dropping of catalogs is implementation dependent and not part of the SQL standard.
- **Schema**: Each catalog can contain multiple schemas. SQL objects such as relations, views, functions and procedures are contained within a schema.
- We can create and drop schemas in a catalog by means of **create schema** and **drop schema** statements.
- To identify a relation uniquely, a three-part name may be used, for example: *UniversityDB.dbo.instructor*
- When a user connects to a database system, the default catalog and schema, which are part of an SQL **environment**, are set up for the connection
- *The catalog and schema can be omitted from* three-part name, in which case the default *catalog and schema are used*


File Edit View Query Project Debug Tools Window Community Help
New Query
EMS
EMS: default database
Object Explorer
Connect
ZHONGHONG-PC (SQL Server 10.50.400
Databases
System Databases
Database Snapshots
EMS
Database Diagrams
Tables
System Tables
dbo.DEPT
dbo.emp
dbo.salgrade
dbo.TestTable1
dbo.TestTable6
test.tt
Views
Synonyms
Programmability
Service Broker
Storage
Security
EMSCUR
houserental
large
Northwind
DBO: default schema
SQLQuery1.sql - Z...C\zhonghong (53))*
select * from emp
select * from UniversityDB.dbo.instructor
select * from instructor
Results
Messages
(18 row(s) affected)
(14 row(s) affected)
Msg 208, Level 16, State 1, Line 3
Invalid object name 'instructor'.
Query completed with errors.
ZHONGHONG-PC (10.50 SP2)
zhonghong-PC\zhonghong...
EMS
00:00:00
32 rows

4.6 Index Creation

- Many queries reference only a small proportion of the records in a table.
- It is inefficient for the system to read every record to find a record with particular value
- An **index** on an attribute(s) of a relation is a data structure that allows the database system to find those tuples in the relation that have a specified value for that attribute(s) efficiently, without scanning through all the tuples of the relation.
- Analogous to
  - A list of goods in a repository
  - Book index cards in an old library
- Cost: slower writes and increased storage space



Index Creation (cont.)

- We create an index with the **create index** command

  create index <name> **on** <relation-name> (attribute-list)

- Example:

  create table *student*
  (*ID* **varchar** (5),
  name **varchar** (20) **not null**,
  dept_name **varchar** (20),
  tot_cred **numeric** (3,0) **default** 0,
  primary key (*ID*));

  create index *studentName_index* **on** *student*(*Name*)

- The query:

  select *
  from *student*
  where *Name* = 'Mary'

  can be executed by using the index to find the required record, without looking at all records of *student*

- A primary key is set as an index.

4.9 Authorization

- Give permissions to **users** to do **certain operations** on **some database objects** such as tables, views, stored procedures, and so on.
- The **grant** statement is used to grant authorization

  grant <privilege list> **on** <relation or view > **to** <user list>
- <user list> is:
  - a user-id
  - **public**, all valid users
  - A role
- Example:
  - **grant select on** *department* **to** Amit, Satoshi

Privileges in SQL

- **select**: allows read access to relation, or the ability to query using the view
- **insert**: the ability to insert tuples
- **update**: the ability to update using the SQL update statement
- **delete**: the ability to delete tuples.

Revoking Authorization in SQL

- The **revoke** statement is used to revoke authorization.

  revoke <privilege list> **on** <relation or view> **from** <user list>

- Example:

  revoke select on *student* **from** U_1, U_2, U_3

- If <user list> includes **public,** all users lose the privilege except those granted it explicitly.

Roles

- Using roles for authorization can greatly reduce the workload of authorization
- A role in a DBMS is a set of authorizations for a specific group of users (e,g, Instructor, Student, Dean, Administrator).
- Users with a certain role inherits all authorizations for that role.
- To create a role we use:

  create role <name>
- Example:

  create role *instructor*
- Once a role is created we can assign “users” to the role using:

  grant <role> **to** <users>
- Example:

  grant *instructor* **to** Amit, Satoshi

Note: Grant <role> to <users> is not supported by SQL Server (although it is standard)

End of Chapter 4